

Programming Massively Parallel Processors

A Hands-on Approach

Fourth Edition

Chapter 3 - Multidimensional grids and data

Exercise

Q1. In this chapter we implemented a matrix multiplication kernel that has each thread produce one output matrix element. In this question, you will implement different matrix-matrix multiplication kernels and compare them.

- a. Write a kernel that has each thread produce one output matrix row. Fill in the execution configuration parameters for the design.

[\[Exercise_Ch3_Q1.cu\]](#)

- b. Write a kernel that has each thread produce one output matrix column. Fill in the execution configuration parameters for the design.

[\[Exercise_Ch3_Q1.cu\]](#)

- c. Analyze the pros and cons of each of the two kernel designs.

[\[Row-fixed\]](#) One thread computes one entire row.

Pros.

1. Simple indexing.

Each thread is responsible for one row, so no complex coordinate transformations are required. This makes the kernel easy to implement and debug.

2. Low warp divergence.

Threads within a warp typically execute the same instructions, resulting in consistent control flow and higher warp execution efficiency.

3. Clear thread-level parallelism.

Rows are independent, so synchronization is generally unnecessary, simplifying the kernel design.

Cons.

1. Poor memory coalescing

Threads within a warp access elements from different rows of matrix $A[\text{row} \cdot K + \text{kk}]$, resulting in strided memory access and inefficient global memory transactions.

2. Excessive global memory traffic

Elements of matrix B are repeatedly loaded from global memory by multiple threads without being shared, leading to excessive memory bandwidth usage.

3. Limited parallelism

The total number of threads is bounded by the number of rows (M). If M is small, the GPU may not be fully utilized, reducing overall performance.

[Col-fixed] One thread computes one entire col.

Pros.

1. Good memory coalescing for matrix B ($B[kk * N + col]$)

2. Low warp divergence

Threads execute identical loops with minimal branching, resulting in consistent control flow and efficient warp execution.

3. Clear thread-level parallelism

Each column is computed independently, so synchronization is generally unnecessary and race conditions are avoided.

Cons.

1. Redundant loads of matrix A ($A[row * K + kk]$)

2. Lack of data reuse

Matrix data is repeatedly fetched from global memory without cooperative sharing between threads.

3. Limited parallelism

Total threads $\approx N$.

4. No shared memory utilization

The kernel relies entirely on global memory and does not exploit shared memory tiling, which is critical for high-performance matrix multiplication.

Q2. A matrix-vector multiplication takes an input matrix B and a vector C and produces one output vector A. Each element of the output vector A is the dot product of one row of the input matrix B and C, that is, $A[i] = \sum_j B[i][j] * C[j]$. For simplicity we will handle only square matrices whose elements are singleprecision floating-point numbers. Write a matrix-vector multiplication kernel and the host stub function that can be called with four parameters: pointer to the output matrix, pointer to the input matrix, pointer to the input vector, and the number of elements in each dimension. Use one thread to calculate an output vector element.

[Exercise_Ch3_Q2.cu]

Q3. Consider the following CUDA kernel and the corresponding host function that calls it:

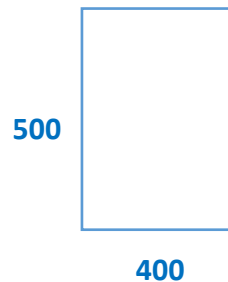
```

01  __global__ void foo_kernel(float* a, float* b, unsigned int M,
unsigned int N) {
02      unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
03      unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
04      if(row < M && col < N) {
05          b[row*N + col] = a[row*N + col]/2.1f + 4.8f;
06      }
07  }
08  void foo(float* a_d, float* b_d) {
09      unsigned int M = 150;
10      unsigned int N = 300;
11      dim3 bd(16, 32);
12      dim3 gd((N - 1)/16 + 1, (M - 1)/32 + 1);
13      foo_kernel <<< gd, bd >>>(a_d, b_d, M, N);
14  }

```

- What is the number of threads per block?
 $16 * 32 = 512$
- What is the number of threads in the grid?
 $16 * 32 * 10 * 10 = 51200$
- What is the number of blocks in the grid?
 $10 * 10 = 100$
- What is the number of threads that execute the code on line 05?
 $150 * 300 = 45000$

Q4. Consider a **2D matrix with a width of 400 and a height of 500**. The matrix is stored as a one-dimensional array. Specify the array index of the matrix element at row 20 and column 10:



- If the matrix is stored in row-major order.
 $20 * 400 + 10 = 8010$
- If the matrix is stored in column-major order.
 $10 * 500 + 20 = 5020$

Q5. Consider a 3D tensor with a width of 400, a height of 500, and a depth of 300. The tensor is stored as a one-dimensional array in row-major order. Specify the array index of the tensor element at $x = 510$, $y = 520$, and $z = 55$.

Out of Bounds